JAVA DSA PROJECT

DATA STRUCTURES & ALGORITHMS

2025 — 2026

SEARCH • SORT • LINK • STACK • HASH • HEAP

Discover your next event destination

Implementing 10+ Data Structures & 7 Sorting/Searching Algorithms on 20 real-world events across 7 Indian cities.

20	7	10+	6
EVENTS	ALGORITHMS	DATA STRUCTS	COs COVERED

Abstract

LocalPulseDSA is a Java application that models a real-world event discovery platform using 20 events across 7 Indian cities. It demonstrates every core DSA concept — from asymptotic analysis and sorting/searching to linked lists, stacks, queues, heaps, hash tables, and Java Collections — applied to practical problems like ticket booking, category filtering, and top-K event selection.

Event ADT | $O(n^2)$ & $O(n \log n)$ | Hash Chaining + OA | Java Collections | Empirical Analysis

System Architecture

Event ADT — Data Model → **Algorithms** — Sort/Search → **Structures** — LL/Stack/Q → **Hash+Heap** — Lookup/Top-K

Java 17+ | OOP + ADT | Arrays | LinkedList | PriorityQueue | HashMap/TreeMap

Asymptotic Analysis — Big-O Summary (CO1)

Linear Search	Binary Search	Bubble Sort	Selection Sort	Insertion Sort	Merge Sort	Quick Sort
$O(n)$	$O(\log n)$	$O(n^2)$	$O(n^2)$	$O(n^2)/O(n)$	$\Theta(n \log n)$	$O(n \log n)$
Space: O(1)	Space: O(1)	Space: O(1)	Space: O(1)	Space: O(1)	Space: O(n)	Space: O(log n)

CO-WISE IMPLEMENTATION

CO1 Asymptotic Analysis & Searching / Sorting

- Linear Search O(n) & Binary Search O(log n) by event ID
- Bubble Sort (price asc) — $O(n^2)$, optimized with swap flag
- Selection Sort (name A–Z) & Insertion Sort (date asc)
- Merge Sort Θ(n log n) by capacity desc; Quick Sort by ID

CO2 Abstract Data Types — Linked List Variants

- Event ADT: encapsulated fields, constructor, toString()
- Singly LL: insert front/end, delete, search city, reverse
- Floyd's cycle detection algorithm on event catalog SLL
- Doubly LL (browsing history) & Circular LL (featured carousel)

CO3 Stacks, Queues, Deques & Heaps

- Array-based Stack: push/pop/peek for undo navigation O(1)
- Circular Queue: FIFO ticket registration, enqueue/dequeue
- EventDeque: addRecent/addUpcoming, recent-event viewer
- Max-Heap (capacity) & Min-Heap PriorityQueue (cheapest first)

CO4 Hash Tables & Java Collections Framework

- Hash Table (Separate Chaining): polynomial hash, city lookup
- Hash Table (Open Addressing / Linear Probing): ID lookup
- HashMap, TreeMap, LinkedHashMap — city grouping & sorting
- HashSet, ArrayDeque, PriorityQueue in live booking workflow

KEY DATA STRUCTURES

Linked Lists
- Singly LL — catalog
- Doubly LL — history
- Circular LL — carousel
- Floyd's cycle detect

Stack & Queue
- Array Stack — undo
- Circular Queue — FIFO
- EventDeque — viewer
- Overflow/Underflow checks

Heap Structures
- Max-Heap — top capacity
- Min-Heap PQ — cheapest
- siftUp / siftDown
- Parent=(i-1)/2 indexing

Hash Tables
- Separate Chaining
- Open Addressing (LP)
- Polynomial rolling hash
- Load factor α < 0.7

Conclusion & Future Scope

This project demonstrates all core DSA concepts — asymptotic analysis, 5 sorting algorithms, 2 search algorithms, 3 linked list types, stack, queue, deque, max/min heap, hash tables with chaining and open addressing, and the Java Collections Framework — integrated into a realistic event management system. Future scope includes graph-based venue routing (Dijkstra/BFS), Trie-based event name autocomplete, and a REST API layer backed by a persistent database.

Name: M. Sai Bhavitha Reddy

Student Id: 2520030551

(DEEMED TO BE UNIVERSITY)